DrawMotion : Real-Time Hand Gesture Tracking and Interactive Drawing System

1.1 Introduction:

1.1.1 Purpose of the System:

The purpose of this document is to define the Software Requirements Specification (SRS) for a web application that allows users to draw on a canvas using their index and middle fingers. The application includes features like selecting drawing colors, shapes, brush sizes, and other functionalities similar to a standard paint application. The system integrates a custom-trained machine learning model to track finger movements without relying on external libraries or frameworks like YOLO, OpenCV, or MediaPipe

1.1.2 Scope of the System:

The application provides a drawing interface powered by a custom-trained machine learning model for finger tracking. It is designed to:

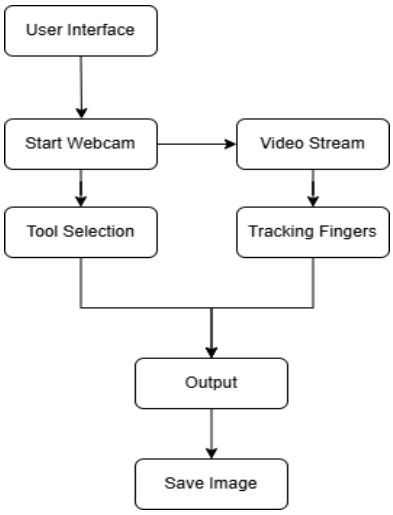
- Enable real-time finger tracking using a proprietary algorithm and model.
- Allow users to draw freehand, create shapes, and select brush sizes and colors.
- Be accessible via a web browser, with the backend built using Python for machine learning and Node.js with Express.js for server-side functionality.

1.2 General Description of the System:

1.2.1 Overall Description:

The application provides a drawing interface powered by a custom-trained machine learning model for finger tracking. It is designed to:

- Enable real-time finger tracking using a proprietary algorithm and model.
- Allow users to draw freehand, create shapes, and select brush sizes and colors.
- Be accessible via a web browser, with the backend built using Python for machine learning and Node.js with Express.js for server-side functionality.



1.2.2 Feasibility Study:

1. **Technical Feasibility:**
   - Uses modern technologies like Python and Node.js.
   - Runs on devices with camera support and modern web browsers.
   - Custom ML models ensure flexibility and independence from external libraries.

2. **Economic Feasibility:**
   - Utilizes open-source tools and technologies to minimize costs.
   - Major expenses include development time and server hosting.

3. **Operational Feasibility:**
   - User-friendly system designed for easy accessibility.
   - Targets a wide audience, including artists, educators, and casual users.

1.3 Functional Requirements:

1.3.1 Module Description:

Video Capture Module
- Captures live video feed from the webcam.
- Adjusts resolution and frame rate for optimal performance.
- Passes frames to processing modules.

Hand Detection and Tracking Module
- Detects and tracks hand movements in real time.
- Identifies key points like fingertips and palm position.
- Provides coordinates for further processing.

Gesture Recognition Module
- Recognizes gestures such as index finger up, fist, or open palm.
- Maps gestures to actions like drawing, erasing, or shape selection.
- Uses custom ML models or heuristic-based classification.

Drawing Module
- Allows users to draw freehand and create shapes.
- Supports brush size, color selection, and stroke thickness.
- Provides undo/redo functionality.

User Interface (UI) Module
- Displays video feed with gesture-based controls.
- Provides buttons for tools like brush, eraser, and shape selector.
- Offers a settings menu for user preferences.

Event Handling Module
- Detects keyboard shortcuts and system events.
- Manages transitions like drawing mode, erasing mode, and clearing canvas.

Custom Algorithm Module

- Implements hand segmentation, contour analysis, and feature extraction.
- Uses in-house algorithms for gesture classification.

Data Management and Analytics Module

- Logs hand positions and gestures for debugging.
- Provides performance metrics and analytics for accuracy improvement.

Save and Export Module

- Saves drawings in PNG, JPEG, or SVG formats.
- Supports exporting drawings with transparent backgrounds.

Undo/Redo Module

- Allows users to undo or redo recent actions.
- Maintains drawing history for better flexibility.

System Optimization and Performance Module

- Ensures real-time gesture recognition with minimal lag.
- Reduces computational overhead for smoother performance.

1.3.2 Functions of Various Users of the System:

End Users

- Interact with the system to draw on the canvas using hand gestures.
- Customize drawings using brush sizes, colors, and shape selection.
- Save or export completed drawings for future use.

Developers

- Enhance gesture recognition accuracy by improving the ML model.
- Add new features and optimize application performance.
- Ensure compatibility across different devices and browsers.

System Administrators

- Maintain backend servers and system performance.
- Manage updates to the machine learning model and other components.
- Address user-reported issues and system failures.

1.4.Non-Functional Requirements:

Performance:

- The system should provide real-time hand tracking with minimal latency.
- Gesture recognition should have a response time of less than 100ms.
- The application should render drawings smoothly without noticeable lag.

Scalability:

- The backend should support multiple concurrent users without performance degradation.
- The system should allow for future expansion, such as adding multi-hand tracking or new gestures.
- Should be able to integrate with AR/VR environments in the future.

Usability:

- The user interface should be intuitive and easy to use without requiring extensive training.
- Users should receive real-time visual feedback for gestures and interactions.
- The system should provide help documentation or tooltips for first-time users.

Security:

- The system should not store user gesture data or drawings without explicit consent.
- All user interactions should be processed locally or securely on the server.
- The application should follow standard web security protocols to prevent unauthorized access.

Compatibility:

- The application should work on modern web browsers like Chrome, Firefox, and Edge.
- The system should support devices with different screen sizes, including PCs, tablets, and smartphones.
- The software should be operating system-independent and work on Windows, macOS, and Linux.

Reliability:

- The system should have 99.5% uptime, ensuring availability for users.
- In case of failure, it should recover quickly without data loss.
- The hand tracking and gesture recognition should work in varying lighting conditions with at least 90% accuracy.

Maintainability:

- The codebase should be modular and well-documented to allow for easy updates.
- Developers should be able to replace or upgrade individual modules without affecting the entire system.
- System logs should be maintained for troubleshooting and performance monitoring.

Portability:

- The application should be deployable on multiple platforms, including local machines and cloud environments.
- The system should allow easy migration between different hardware and software setups.

1.5 Interface Requirements:

1.5.1 GUI

- The application provides a responsive and intuitive user interface.
- The canvas is the main interaction area, occupying the majority of the screen.
- Tools such as the brush, eraser, shape selector, and color palette are available as buttons or dropdowns around the canvas.
- A real-time preview of selected brush size and color is displayed.
- Gesture-based controls are visually indicated on the interface.
- A settings menu is available for configuring preferences such as brush sensitivity or gesture calibration.

1.5.2 Hardware Interface

- A camera-enabled device (e.g., laptop, smartphone, or tablet) is required for finger tracking.
- Devices must support HD video input for optimal tracking performance.
- Minimum hardware specifications:
    - Processor: Dual-core CPU, 2.0 GHz or higher
    - RAM: 4 GB or higher
    - Camera resolution: 720p or higher

1.5.3 Software Interface

- The application is compatible with modern web browsers supporting HTML5, JavaScript, and APIs for accessing camera feeds.
- The backend interacts with:
    - Custom ML Model: Implemented in Python, deployed on the server, and integrated via APIs.
    - Node.js Server: Handles client requests, communicates with the ML model, and manages real-time tracking data.
- The frontend is built with HTML, CSS, and JavaScript frameworks.
- Third-party libraries or APIs (if any) must comply with standard web security protocols.

1.6 Data Dictionary:

1. Table Name : Login_Table

Description: Stores login credentials for Users.

Field Name	Data Type	Description	Constraints
user_id	Integer	Unique identifier for each user	Primary Key, Auto Inc
password	String	Password of the account	6-12 characters
username	String	Username of the user	Unique, Max 50 characters
email	String	Email address of the user	Unique, Valid Email Format
role	String	Role of the user (End User, Admin)	Predefined Values
created_at	Timestamp	Account creation date and time	Auto-generated

2. Table Name : DrawingDetails_Table

Description: Stores the details of drawing of users.

Field Name	Data Type	Description	Constraints
Drawing_id	Integer	Unique identifier for each drawing	Primary Key, Auto Increment
user_id	Integer	Identifier of the user being tracked	Foreign Key (User Table)
Image_src	String	Stores the location of drawing.	Any path String
Created_at	Timestamp	Time of creation	Auto-generatred

.

1.7 Diagrams

1.7.1 Use Case Diagram

Use case diagrams are a common way to communicate the major functions of a software system. A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved. A use case diagram can identify the different types of users of a system and the different use cases and will often be accompanied by other types of diagrams as well.

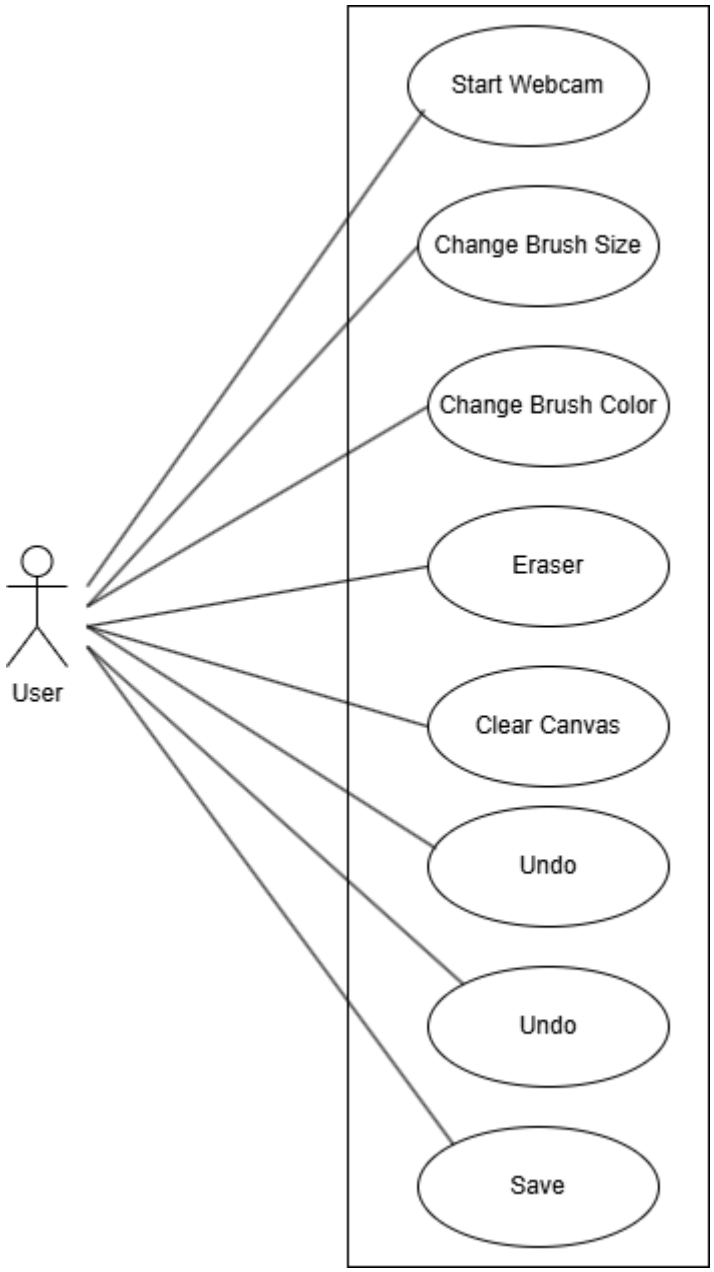
Use cases are nothing but the system functionalities written in an organized manner. Now another thing which is relevant to the use cases are the actors. Actors can be defined as something that interacts with the system.

So in brief, the purposes of use case diagrams can be as follows:

- Used to gather requirements of a system.
- Used to get an outside view of a system.
- Identify external and internal factors influencing the system.
- Show the interacting among the requirements are actors.

Some Symbols Used are:

Use Case Diagram of the System:



1.7.2 Class Diagram

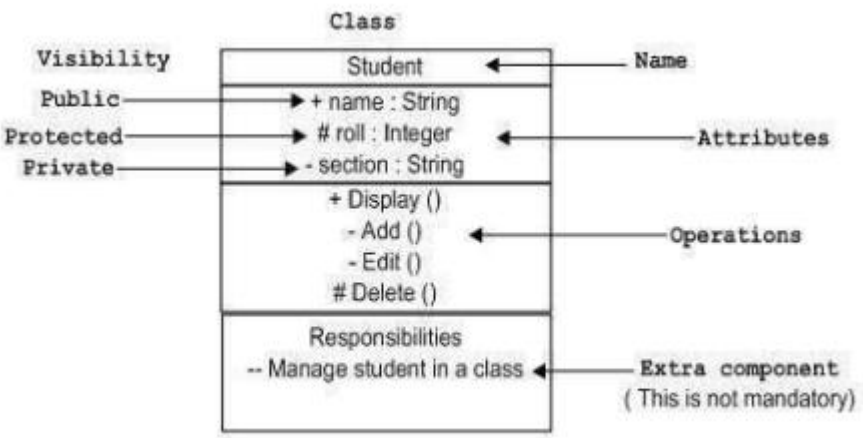
The class diagram is the main building block of object-oriented modelling. It is used both for general conceptual modelling of the systematics of the application, and for detailed modelling translating the models into programming code. Class diagrams can also be used for data modeling. The classes in a class diagram represent both the main elements, interactions in the application, and the classes to be programmed.

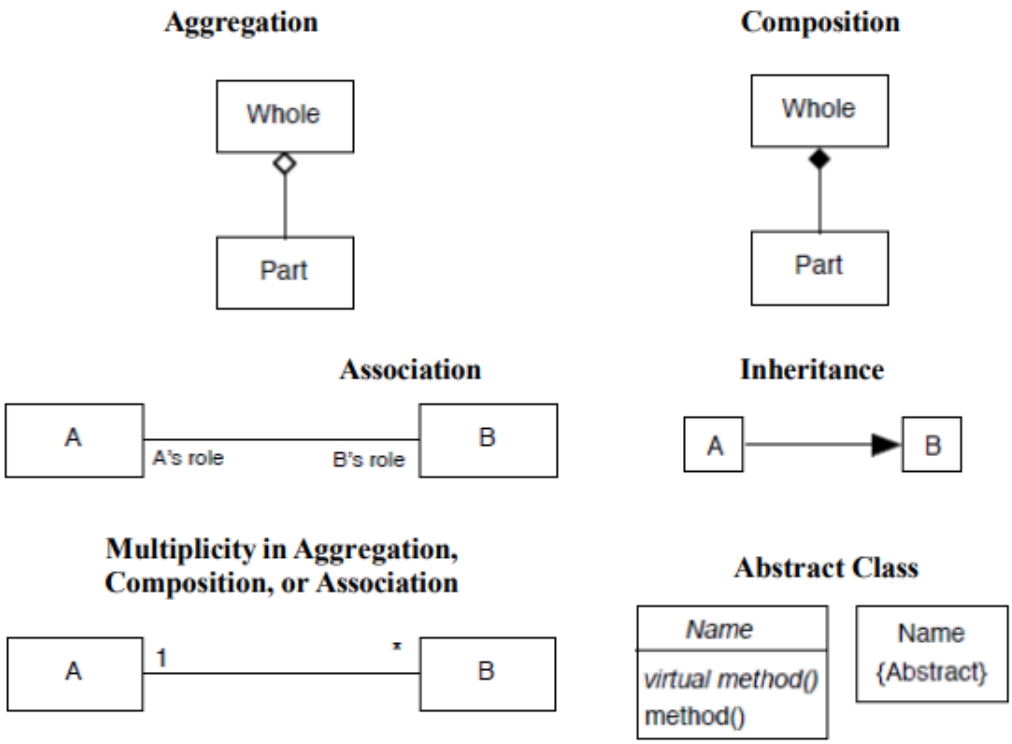
In the diagram, classes are represented with boxes that contain three compartments:

- The top compartment contains the name of the class. It is printed in bold and centered, and the first letter is capitalized.
- The middle compartment contains the attributes of the class. They are left-aligned and the first letter is lowercase.
- The bottom compartment contains the operations the class can execute.

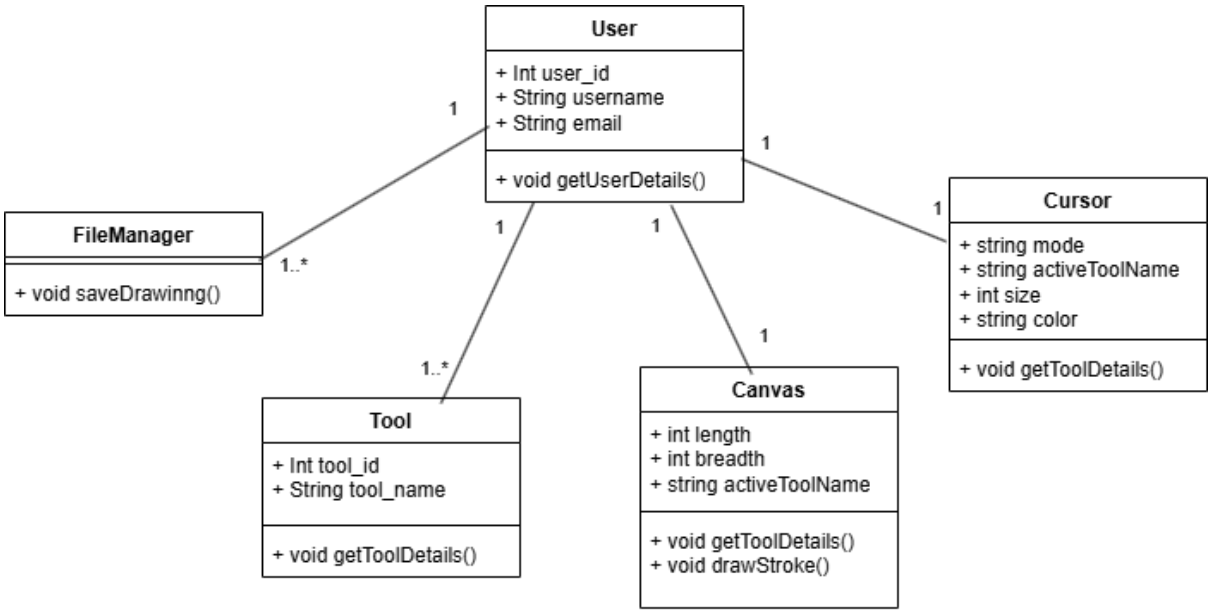
They are also left- aligned and the first letter is lowercase. In the design of a system, a number of classes are identified and grouped together in a class diagram that helps to determine the static relations between them. With detailed modelling, the classes of the conceptual design are often split into a number of subclasses.

Symbols used are:





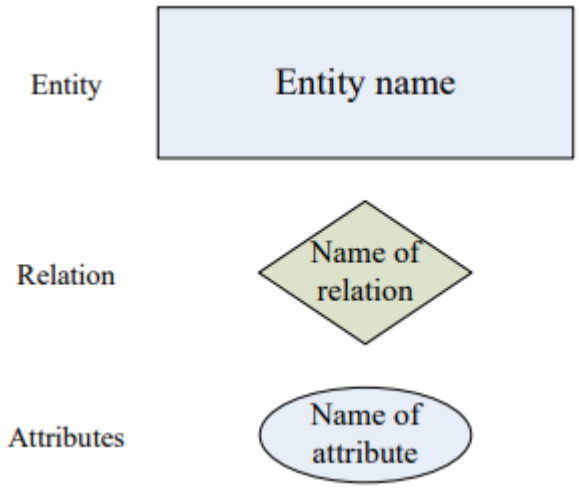
Class Diagram of the System:



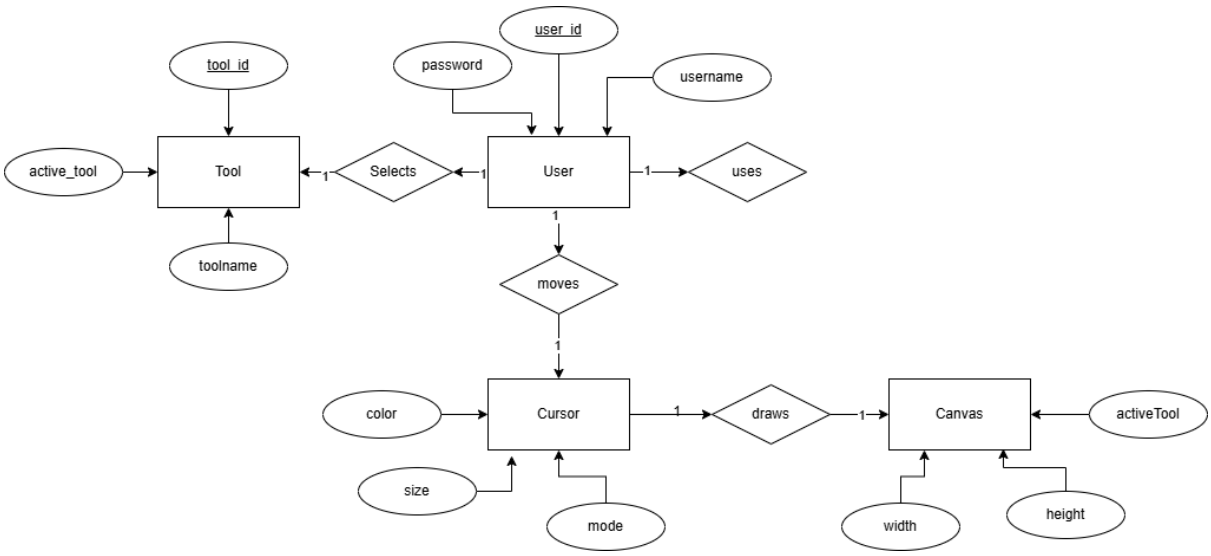
1.7.3 ER Diagram

Entity-Relationship model is used to represent a logical design of a database to be created. In ER model, real world objects (or concepts) are abstracted as entities, and different possible associations among them are modeled as relationships. We represents the attributes, entities and relation using the ER diagram. Using this ER diagram, table structures are created, along with required constraints. Finally, these tables are normalized in order to remove redundancy and maintain data integrity. Thus, to have data stored efficiently, the ER diagram is to be drawn as much detailed and accurate as possible.

Symbols used in ER diagram:



ER Diagram of the System:

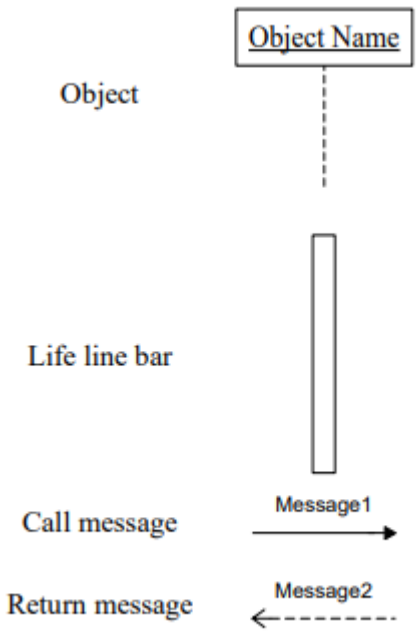


1.7.4 Sequence Diagram

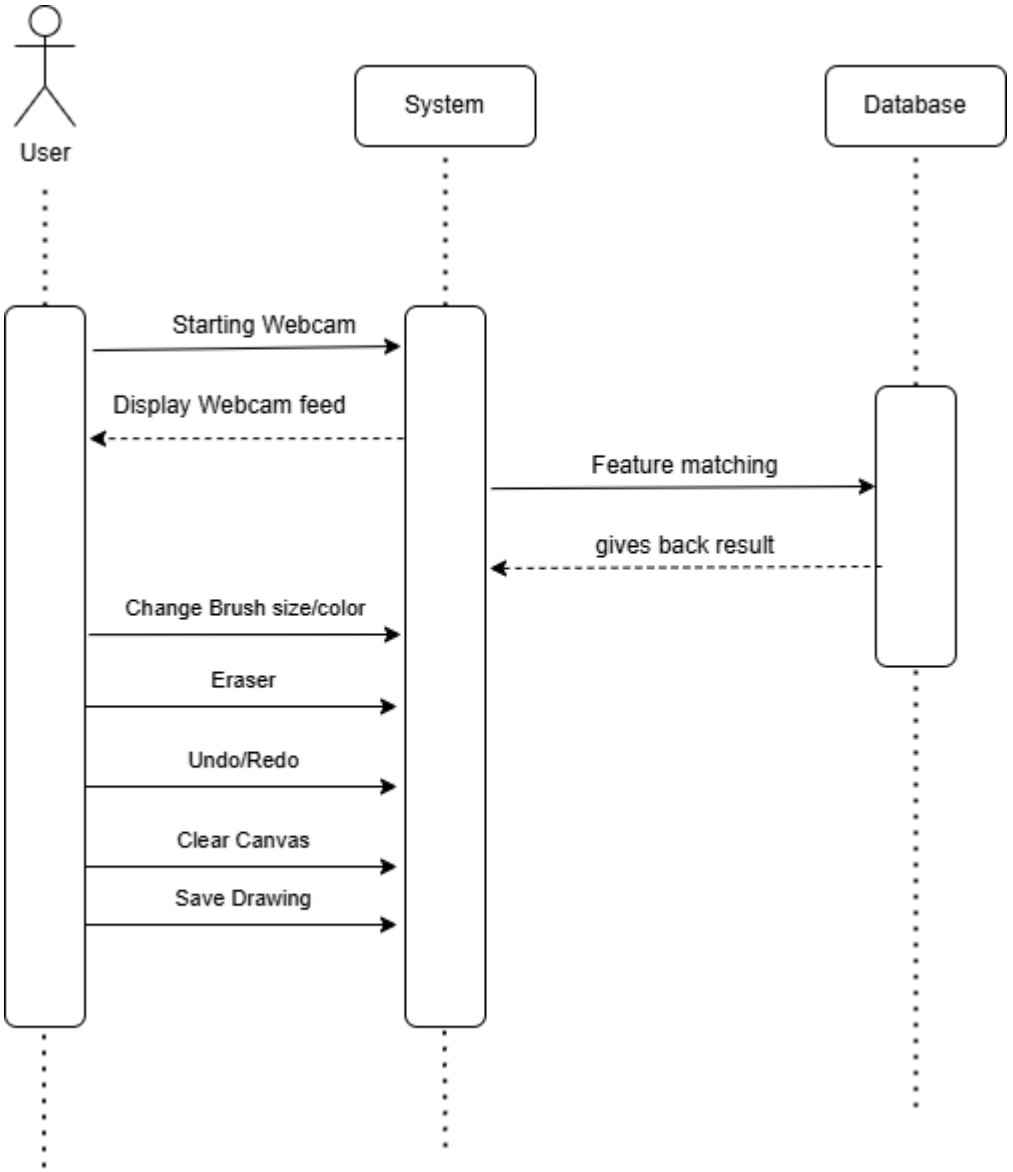
Sequence diagram represents the behavioural aspects of a system. Sequence diagram shows the interactions between the objects by means of passing messages from one object to another with respect to time in a system.

Sequence diagram contains the objects of a system and their life-line bar and the messages passing between them. Objects appear at the top portion of sequence diagram. Object is shown in a rectangle box. Name of object precedes a colon ':' and the class name, from which the object is instantiated. The whole string is underlined and appears in a rectangle box. A down-ward vertical line from object-box is shown as the life-line of the object. A rectangle bar on life-line indicates that it is active at that point of time. Messages are shown as an arrow from the life-line of sender object to the life-line of receiver object and labelled with the message name.

Symbols used in Sequence diagram:



Sequence Diagram of the System:



1.7.5 State Diagram

State diagram is one of the UML diagrams used to model dynamic nature of a system. State diagram describes the flow of control from one state to another state.

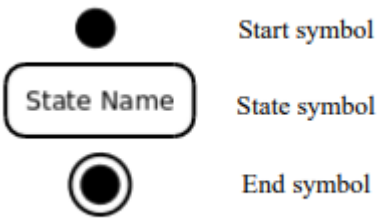
States are defined as a condition in which an object exists and it changes when some event is triggered. So the most important purpose of State diagram is to model life time of an object from creation to termination.

State diagrams are also used for forward and reverse engineering of a system. But the main purpose is to model reactive system. Reactive systems can be defined as a system that responds to external or internal events.

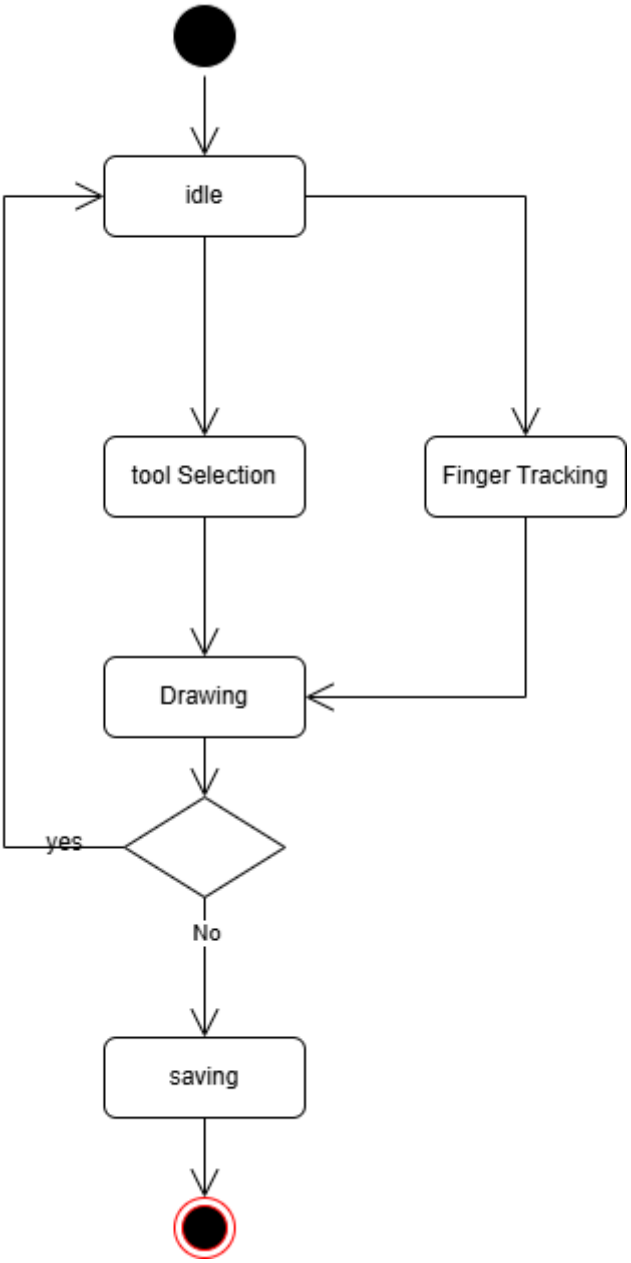
Following are the main purposes of using State diagrams:

- To model dynamic aspect of a system.
- To model life time of a reactive system.
- To describe different states of an object during its life time.
- Define a state machine to model states of an object

Symbols used in State diagram:



State Diagram of the System:

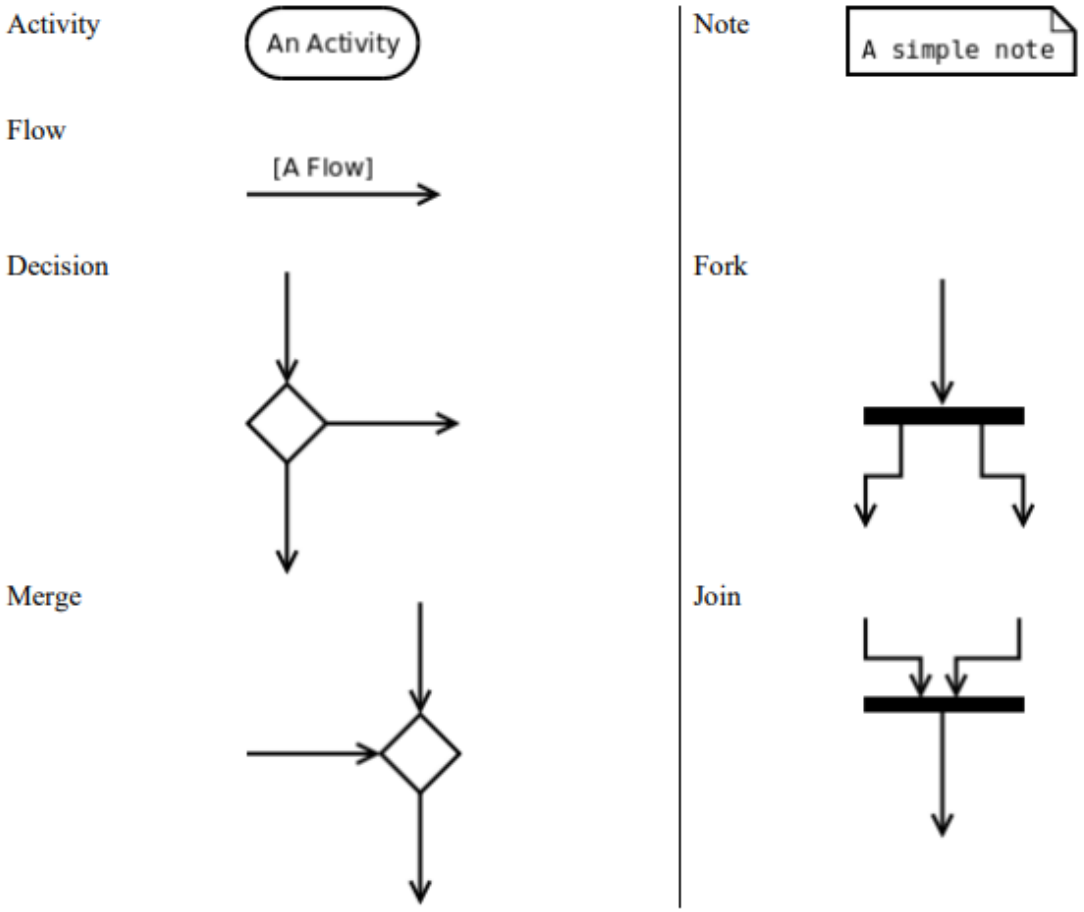
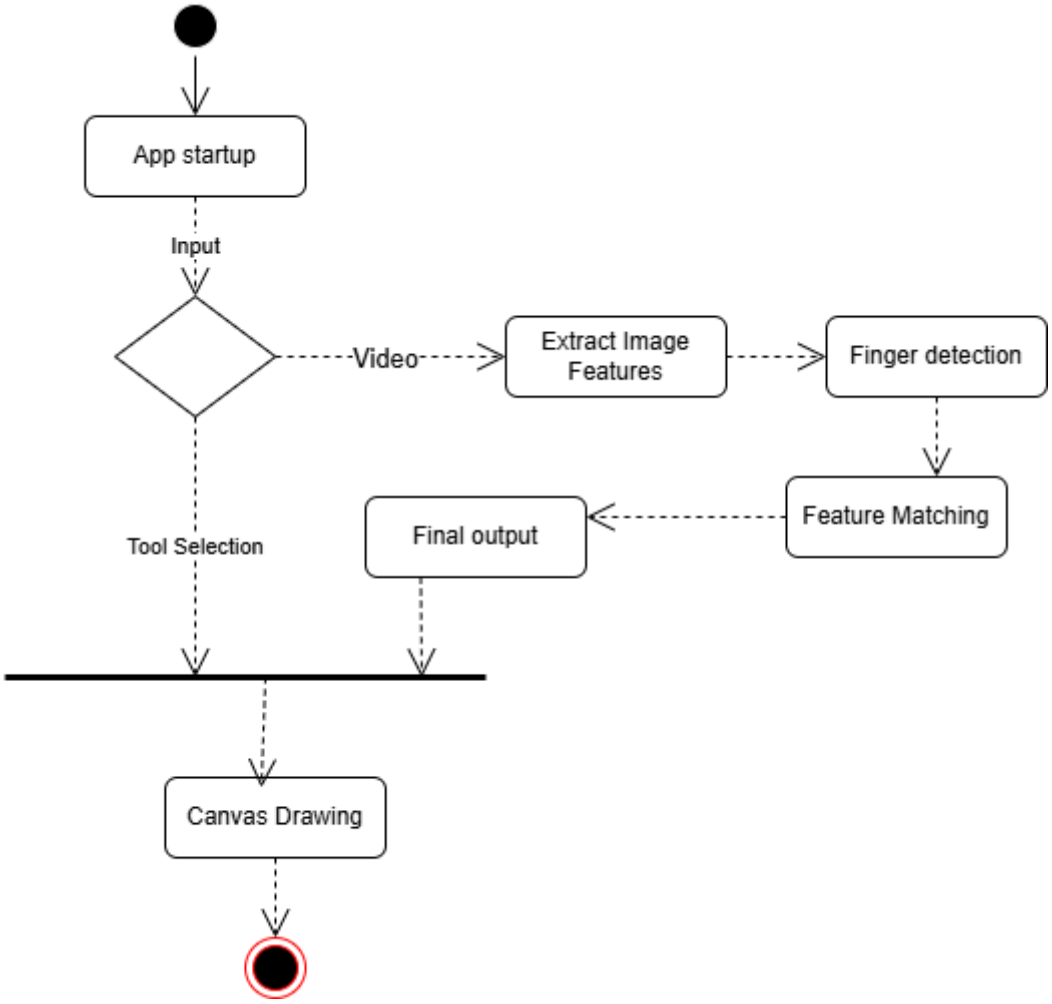


1.7.6 Activity Diagram

An activity denotes a particular action taken in the logical flow of control. This could simply be invocation of a mathematical function, alter an object's properties and so on. An activity is represented with a rounded rectangle, as shown in figure. A label inside the rectangle identifies the corresponding activity.

There are two special types of activity nodes: initial and final. They are represented with a filled circle, and a filled in circle with a border respectively. Initial node represents the starting point of a flow in an activity diagram. There could be multiple initial nodes, which mean that invoking that particular activity diagram would initiate multiple flows. A final node represents the end point of all activities. Like an initial node, there could be multiple final nodes. Any transition reaching a final node would stop all activities. A flow is represented with a directed arrow. A decision node, represented with a diamond, is a point where a single flow enters and two or more flows leave. This is represented with a diamond shape, with two or more flows entering, and a single flow leaving out. Fork is a point where parallel activities begin. A join is depicted with a black bar, with multiple input flows, but a single output flow. Physically it represents the synchronization of all concurrent activities.

Symbols used in Activity diagram:



Activity Diagram of the System: